Snare Locations

John D. Robinson

2025-09-23

Contents

Formatting Snare Locations **1**

Load files 1

Convert to UTM 2

Create `oper` and `traplocs` 7

Create `translatable` 8

Create `CGAsnarebuff` 10

Formatting Snare Locations

Here, we will create spatial objects with information on the location of hair snares (*i.e.*, traps) associated with the project. There are several input files needed...

1. `input/Hair Snare Summaries (Both Years).xlsx` - this is the actual Excel spreadsheet with information on landowners, locations, and weekly check dates for all hair snares in each of the two years of the project
2. `input/HS23_coords.csv` - Pulling a few columns out of the file above - 2023 tab
3. `input/HS24_coords.csv` - Pulling a few columns out of the file above - 2024 tab

The primary outputs will include...

1. `1_snareLocations.pdf` - a markdown (in PDF format) that walks through the steps
2. `snareLocations.Rda` - an `.Rda` file with two objects
   - `locs` - coordinates for all unique hair snares associated with the project (a $J \times 3$ data frame)
   - `oper` - operational status of each hair snare in each of the two years (a $J \times 2$ data frame)
3. `translatable.csv` - a table with translation information for the naming of hair snares in the project
4. `CGAsnarebuff` - a polygon shapefile defining a 4km buffer around the snare array (may not be used for SCR models)

Load files

Read the `.csv` files with coordinates into R. Remove rows associated with snares *NOT* operated in a given year (these will have no information - " " - in the `Date.Baited` column)...

```r
HS23 <- read.csv("../input/HS23_coords.csv", header = TRUE)
HS24 <- read.csv("../input/HS24_coords.csv", header = TRUE)
sum(!is.na(HS23$Longitude) & HS23$Date.Baited != "")
```

```
## [1] 130
```

```r
sum(!is.na(HS24$Longitude) & HS24$Date.Baited != "")
```

```
## [1] 138
```

```r
HS23 <- HS23[HS23$Date.Baited != "",]
HS24 <- HS24[HS24$Date.Baited != "",]

dim(HS23)
```

```
## [1] 130   6
```

```r
dim(HS24)
```

```
## [1] 138   6
```

Convert to UTM

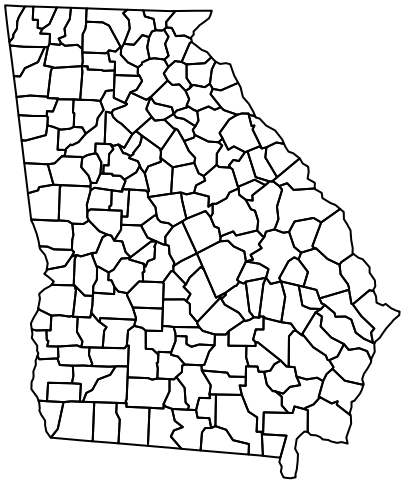
Now convert to UTM projection, which we'll use for all spatial objects.

Note that `crs=32617` is the coordinate reference system for UTM zone 17N (between 78°W and 84°W, our study region sits around 83.5°W) and `crs=4326` is for WGS84 lat/longs.

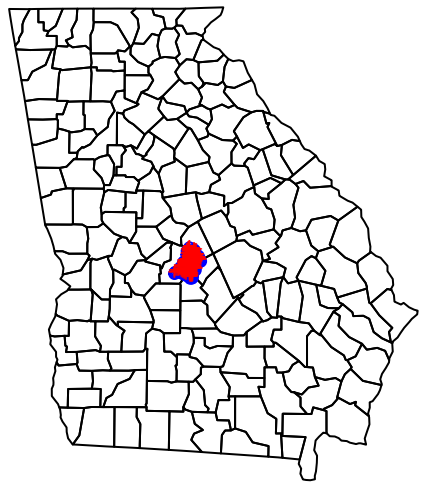
```r
library(sf)
```

```
## Linking to GEOS 3.13.1, GDAL 3.11.0, PROJ 9.6.2; sf_use_s2() is TRUE
```

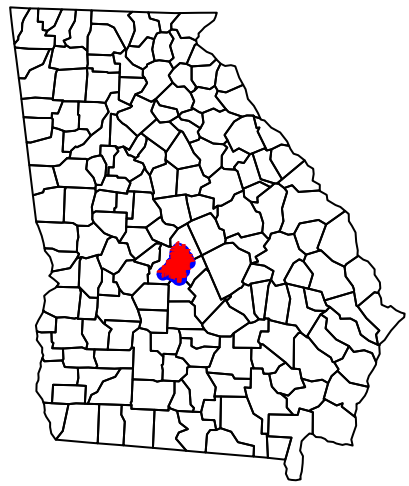
```r
library(usmap)

#Create a UTM map of GA counties using the usmap R package
gamap <- us_map(regions="counties", include="GA")
gamap <- st_transform(gamap, crs=4326) #Convert to WGS84
gamap_utm <- st_transform(gamap, crs=32617) #Convert to UTM
plot(st_geometry(gamap_utm))
```



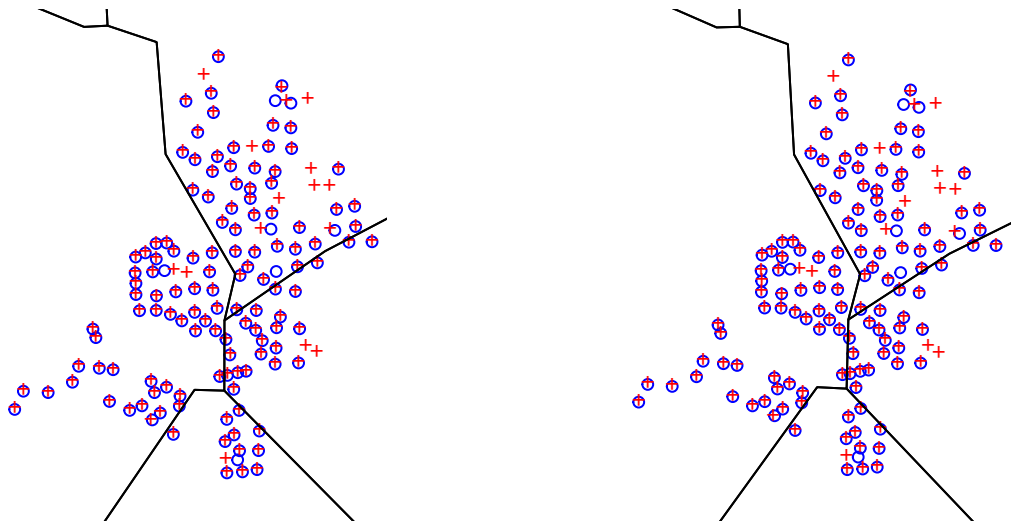
```r
#Convert to sf objects
hs23_sf <- st_as_sf(x=HS23[,c(4,3)], coords = c("Longitude","Latitude"), crs= 4326)
hs24_sf <- st_as_sf(x=HS24[,c(4,3)], coords = c("Longitude","Latitude"), crs= 4326)
plot(st_geometry(gamap))
plot(st_geometry(hs23_sf), pch=1, col="blue", add=TRUE, cex = 0.75)
plot(st_geometry(hs24_sf), pch="+", col="red", add=TRUE, cex = 0.75)
```



```r
#Transform to UTM coordinates
#32617 is the epsg for UTM zone 17N
hs23_utm <- st_transform(hs23_sf, crs=32617)
hs24_utm <- st_transform(hs24_sf, crs=32617)

#Check the plot
plot(st_geometry(gamap_utm))
plot(st_geometry(hs23_utm), pch=1, col="blue", add=TRUE, cex = 0.75)
plot(st_geometry(hs24_utm), pch="+", col="red", add=TRUE, cex = 0.75)
```



```r
#Plot it backwards to double check fine-scale positioning
#And with WGS and UTM next to one another to double check everything
par(mfrow = c(1,2))
#WGS84
plot(st_geometry(hs23_sf), pch=1, col="blue", cex = 0.75)
plot(st_geometry(hs24_sf), pch="+", col="red", add=TRUE, cex = 0.75)
plot(st_geometry(gamap), add=TRUE)
#UTM
plot(st_geometry(hs23_utm), pch=1, col="blue", cex = 0.75)
plot(st_geometry(hs24_utm), pch="+", col="red", add=TRUE, cex = 0.75)
plot(st_geometry(gamap_utm), add=TRUE)
```



Now let's create some unique names (based on UTM coordinates, as in Hooker et al. (2020)) for the traps.

```
#Create unique names based on the UTMcoords for each trap
tmp23 <- st_coordinates(hs23_utm)
names23 <- paste0(round(tmp23[,"X"]),"x",round(tmp23[,"Y"]))
rownames(tmp23) <- names23
tmp24 <- st_coordinates(hs24_utm)
names24 <- paste0(round(tmp24[,"X"]),"x",round(tmp24[,"Y"]))
rownames(tmp24) <- names24
allnames <- unique(c(names23,names24))
length(allnames)
```

```
## [1] 154
```

```
traplocs <- data.frame(East=rep(NA,length(allnames)), North=rep(NA,length(allnames)))
oper <- data.frame(Yr2023=rep(NA,length(allnames)), Yr2024=rep(NA,length(allnames)))
rownames(traplocs) <- allnames
rownames(oper) <- allnames

#Check to make sure there are never mismatches in locations when the UTM name matches
bothyrs <- which(allnames %in% rownames(tmp23) & allnames %in% rownames(tmp24))
checknames <- allnames[bothyrs]
cnt <- 0 #Counter for matches
for(x in 1:length(checknames)) {
    if(tmp23[,"X"][rownames(tmp23) == checknames[x]] == tmp24[,"X"][rownames(tmp24) == checknames[x]]
       & tmp23[,"Y"][rownames(tmp23) == checknames[x]] == tmp24[,"Y"][rownames(tmp24) == checknames[x]]
```

```r
        cnt <- cnt+1 #Add 1 to counter if the X and Y coords match when the trap name matches
    }
}
length(checknames)
```

```
## [1] 114
```

```r
cnt
```

```
## [1] 114
```

```r
#So they all match up perfectly... good
```

Create oper and traplocs

Now fill in the oper and traplocs objects that will be used for modeling.

```r
#Now fill in the contents of traplocs and oper
for(j in 1:length(allnames)) {
  #If the trap name is in both years...
    if(allnames[j] %in% rownames(tmp23) & allnames[j] %in% rownames(tmp24)) {
        traplocs[j,] <- tmp23[which(rownames(tmp23) == allnames[j]),]
        oper[j,] <- c(1,1)
    #If only on 2023
    } else if (allnames[j] %in% rownames(tmp23)) {
        traplocs[j,] <- tmp23[which(rownames(tmp23) == allnames[j]),]
        oper[j,] <- c(1,0)
    #If only in 2024
    } else if(allnames[j] %in% rownames(tmp24)) {
        traplocs[j,] <- tmp24[which(rownames(tmp24) == allnames[j]),]
        oper[j,] <- c(0,1)
    }
}

#Now check your work...
head(traplocs)
```

```
##                     East   North
## 248187x3586614 248187.0 3586614
## 255814x3587061 255814.0 3587061
## 248920x3588050 248919.6 3588050
## 250904x3587890 250904.0 3587890
## 252875x3588670 252874.7 3588670
## 253409x3589929 253409.2 3589929
```

```r
head(oper)
```

```
##                 Yr2023 Yr2024
## 248187x3586614      1      1
## 255814x3587061      1      1
```

```
## 248920x3588050      1      1
## 250904x3587890      1      1
## 252875x3588670      1      1
## 253409x3589929      1      1
```

```r
#These should be 0... no rownames shared when oper == 0
sum(rownames(traplocs)[oper$Yr2023 == 0] %in% rownames(tmp23))
```

```
## [1] 0
```

```r
sum(rownames(traplocs)[oper$Yr2024 == 0] %in% rownames(tmp24))
```

```
## [1] 0
```

```r
#Checking the length of the objects vs. sum of oper and matching rownames
#2023
length(HS23$Site.Name)
```

```
## [1] 130
```

```r
sum(oper$Yr2023)
```

```
## [1] 130
```

```r
sum(rownames(traplocs)[oper$Yr2023 == 1] %in% rownames(tmp23))
```

```
## [1] 130
```

```r
#2024
length(HS24$Site.Name)
```

```
## [1] 138
```

```r
sum(oper$Yr2024)
```

```
## [1] 138
```

```r
sum(rownames(traplocs)[oper$Yr2024 == 1] %in% rownames(tmp24))
```

```
## [1] 138
```

Create `translatable`

Now create a object that we can use to translate between the various names used for hair snares in the dataset.

```r
#Now create translatable
translatable <- data.frame(UTMname=rep(NA, length(allnames)),
                           name23 = rep(NA,length(allnames)),
                           name24 = rep(NA, length(allnames)),
                           oper23 = rep(NA,length(allnames)),
                           oper24 = rep(NA,length(allnames)),
                           lat=rep(NA,length(allnames)),
                           long=rep(NA,length(allnames)),
                           east=rep(NA,length(allnames)),
                           north=rep(NA,length(allnames)))
translatable$UTMname <- allnames
for(x in 1:length(allnames)) {
  tmpcoords23 <- tmpcoords24 <- c()
    if(allnames[x] %in% rownames(tmp23)) {
        translatable$name23[x] <- HS23$Site.Name[which(rownames(tmp23) == allnames[x])]
        tmpcoords23 <- c(HS23$Latitude[which(rownames(tmp23) == allnames[x])],
                         HS23$Longitude[which(rownames(tmp23) == allnames[x])])
    }
    if(allnames[x] %in% rownames(tmp24)) {
        translatable$name24[x] <- HS24$Site.Name[which(rownames(tmp24) == allnames[x])]
        tmpcoords24 <- c(HS24$Latitude[which(rownames(tmp24) == allnames[x])],
                         HS24$Longitude[which(rownames(tmp24) == allnames[x])])
    }
  translatable$east[x] <- traplocs[which(rownames(traplocs) == translatable$UTMname[x]),"East"]
  translatable$north[x] <- traplocs[which(rownames(traplocs) == translatable$UTMname[x]),"North"]
  if(length(tmpcoords23) > 0 & length(tmpcoords24) > 0) {
    if(sum(tmpcoords23 == tmpcoords24)==2) {
      translatable$lat[x] <- tmpcoords23[1]
      translatable$long[x] <- tmpcoords23[2]
    } else {
      stop(paste("Error for",allnames[x],"coords in 2023 and 2024 don't match!!"))
    }
  } else {
    if(length(tmpcoords23) > 0) {
      translatable$lat[x] <- tmpcoords23[1]
      translatable$long[x] <- tmpcoords23[2]
    } else if(length(tmpcoords24) > 0) {
      translatable$lat[x] <- tmpcoords24[1]
      translatable$long[x] <- tmpcoords24[2]
    }
  }
  translatable$oper23[x] <- oper[which(rownames(oper)==allnames[x]),1]
  translatable$oper24[x] <- oper[which(rownames(oper)==allnames[x]),2]
  rm(tmpcoords23, tmpcoords24)
}

head(translatable)
```

```
##          UTMname name23 name24 oper23 oper24      lat      long     east
## 1 248187x3586614 HS 001 HS 001      1      1 32.38816 -83.67687 248187.0
## 2 255814x3587061 HS 002 HS 002      1      1 32.39388 -83.59598 255814.0
## 3 248920x3588050 HS 003 HS 003      1      1 32.40127 -83.66947 248919.6
## 4 250904x3587890 HS 004 HS 004      1      1 32.40027 -83.64835 250904.0
```

```
## 5 252875x3588670 HS 005 HS 005      1      1 32.40774 -83.62762 252874.7
## 6 253409x3589929 HS 006 HS 006      1      1 32.41920 -83.62227 253409.2
##     north
## 1 3586614
## 2 3587061
## 3 3588050
## 4 3587890
## 5 3588670
## 6 3589929
```

```
#A few checks before saving the output so far...
#Any cases where the name doesn't match (i.e., same name, different trap?)
sum(!translatable$name23 == translatable$name24, na.rm = TRUE)
```

```
## [1] 0
```

```
#How many traps not operated in both years
sum(is.na(translatable$name23 == translatable$name24))
```

```
## [1] 40
```

```
table(rowSums(oper)) #114 traps remain in the same spot, 40 are new or moved
```

```
##
##   1   2
##  40 114
```

```
#There are three options, 10, 01, and 11 for operation over the 2 years
table(apply(oper,1,function(x) paste0(x,collapse="")))
```

```
##
##  01  10  11
##  24  16 114
```

```
#114 traps operated in both years, 16 in 2023 but not 2024, 24 in 2024 but not 2023

locs <- traplocs
translatable$name24 <- gsub("B","",translatable$name24)

write.csv(translatable, "translatable.csv", quote=FALSE, row.names=FALSE)
save(oper, locs, file="snareLocations.Rda")
```

Create `CGAsnarebuff`

Now create a buffered polygon that surrounds the trapping array. This will be the state space for the analysis!

For now, we'll create a polygon with a **buffer of 4km**. Note that this was the size of the starting buffer in Hooker et al. (2020).

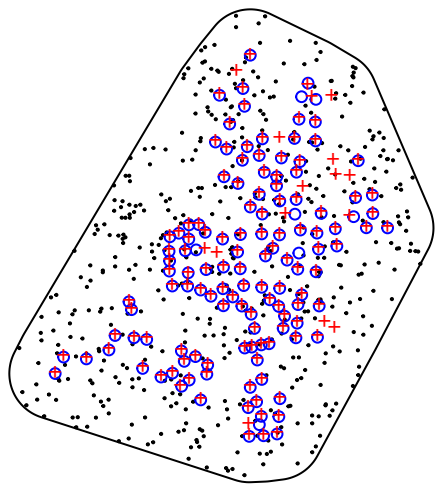
```r
library(shapefiles)
```

```
## Loading required package: foreign

##
## Attaching package: 'shapefiles'

## The following objects are masked from 'package:foreign':
##
##     read.dbf, write.dbf
```

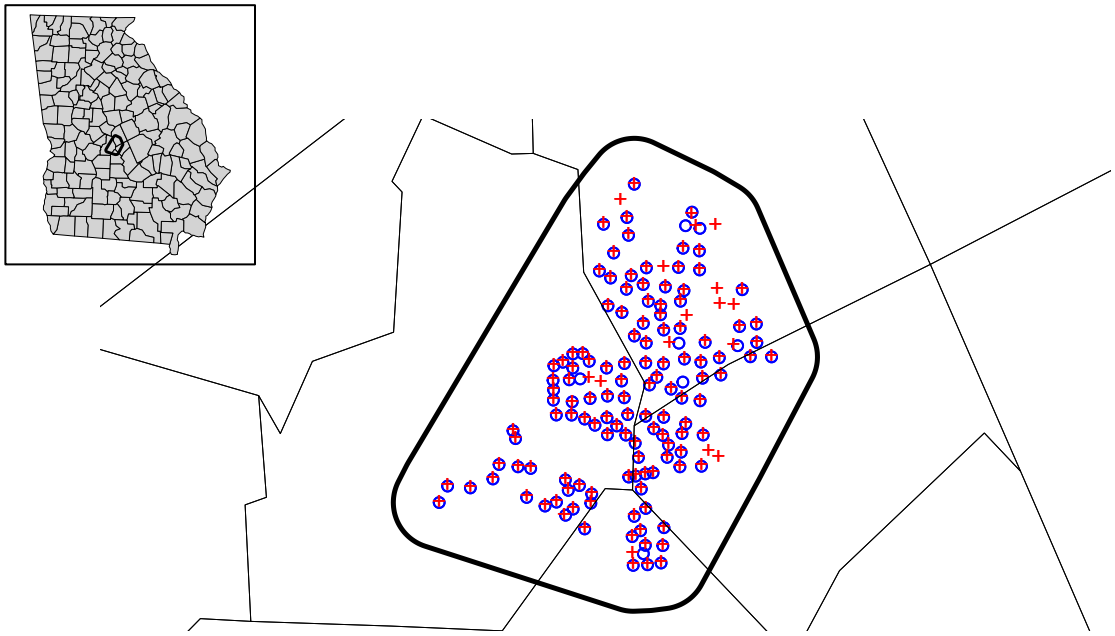
```r
buffer <- 4000 #Buffer setting, in meters
tr <- traplocs

#Make a polygon from the trap locations
p1 <- st_polygon(list(as.matrix(rbind(tr,tr[1,]))))
#identify the points that form the convex hull of p1
p1ch <- p1[[1]][chull(p1[[1]]),]
p1ch_closed <- st_polygon(list(as.matrix(rbind(p1ch,p1ch[1,]))))

#Add the buffer to the convex hull
bp1 <- st_buffer(p1ch_closed,buffer)

#Extract coordinates from the st object and build a shapefile
bp1.coords <- bp1[[1]]
shpdf <- data.frame(Id=rep(1,nrow(bp1.coords)), X=bp1.coords[,1], Y=bp1.coords[,2])
shptbl <- data.frame(Id = 1)
shpfl <- convert.to.shapefile(shpdf, shptbl, "Id", 5)
system("mkdir CGAsnarebuff")
write.shapefile(shpfl, "CGAsnarebuff/CGAsnarebuff")
CGA <- st_read("CGAsnarebuff")
```

```
## Reading layer `CGAsnarebuff' from data source
##   `/scratch/jdrobins/CGA_BlackBears/sampleinfo/CGAsnarebuff'
##   using driver `ESRI Shapefile'
## Simple feature collection with 1 feature and 1 field
## Geometry type: POLYGON
## Dimension:     XY
## Bounding box:  xmin: 244187 ymin: 3577135 xmax: 281169.6 ymax: 3618369
## CRS:           NA
```

```r
plot(st_geometry(CGA))
S1 <- st_sample(CGA, size = 500)
plot(S1, add=TRUE, pch = 19, cex=0.15)
points(traplocs[oper[,1] == 1,], pch=1, col="blue", cex=0.75)
points(traplocs[oper[,2] == 1,], pch="+", col="red", cex=0.75)
```



```r
#Just as a reminder, here's the original trap array with county lines included
#Shouldn't be able to tell that points are overlaid here...
plot(st_geometry(CGA), lwd = 2.5)
plot(st_geometry(hs23_utm), pch=1, col="blue", add=TRUE, cex = 0.75)
points(traplocs[oper[,1] == 1,], pch=1, col="blue", cex=0.75)
plot(st_geometry(hs24_utm), pch="+", col="red", add=TRUE, cex = 0.75)
points(traplocs[oper[,2] == 1,], pch="+", col="red", cex=0.75)
plot(st_geometry(gamap_utm), lwd = 0.25, add=TRUE)
par(fig=c(0.05,0.25,0.65,0.95), mar=c(0,0,0,0), new=TRUE)
plot(st_geometry(gamap_utm), col="light grey", lwd = 0.25)
plot(st_geometry(CGA), add=TRUE, lwd = 1.5)
box(which="figure", lwd=1)
```



So at this point we have our traplocs and oper objects (which will be used directly in the SCR model) and a `CGAsnarebuff` polygon that we can use to mask unsuitable habitat. Now we can move on to matching the samples back to the hair snares where they were collected!